SECURITY_ARCHITECTURE

Helix OTA — Security Architecture Document

> **Document ID:** `HELOTA-SEC-001` **Version:** 1.0.0 **Status:** Active **Last Updated:** 2026-03-05 **Classification:** CONFIDENTIAL — Internal Engineering Use Only **Constitution Reference:** HelixConstitution v1 §1–§4 **Target Platform:** Android 15 on Orange Pi 5 Max (RK3588)

Table of Contents

---

1. Security Overview

1.1 Purpose

This document defines the complete security architecture for the Helix OTA system — an enterprise-grade Over-The-Air update platform delivering signed, verified firmware to fleets of Android 15 devices running on Orange Pi 5 Max (RK3588). Given that a compromised OTA channel can brick devices, exfiltrate data, or grant attackers persistent root-level access to every device in a fleet, security is not a feature of this system — it is the foundation upon which every other feature is built.

The OTA server sits at the nexus of three trust boundaries: (1) the internet-facing API consumed by remote devices, (2) the admin dashboard used by operators to manage fleet updates, and (3) the artifact pipeline that ingests, signs, and distributes

firmware binaries. A breach at any of these boundaries can cascade into catastrophic fleet-wide compromise. This document enumerates the threats, defines the controls, and provides concrete implementation guidance for every security layer.

1.2 Defense-in-Depth Philosophy

Helix OTA implements a defense-in-depth strategy with seven concentric layers. No single control is assumed to be sufficient; every critical security property is enforced by at least two independent mechanisms so that the failure of one control does not result in a security breach.

Layer	Control	Example
L1 — Network	TLS 1.3, mTLS, firewall rules	Encrypted channel prevents eavesdropping
L2 — Identity	Device certificates, JWT, TOTP 2FA	Verified identity before any operation
L3 — Authorization	RBAC, API key scopes	Least-privilege access to resources
L4 — Payload Integrity	RSA-4096 signing, SHA-256 hashing	Artifact tampering detected and rejected
L5 — Boot Chain	AVB, dm-verity, A/B partitions	Compromised partition cannot boot
L6 — Runtime	Container hardening, input validation, rate limiting	Server-side attack surface minimized
L7 — Audit & Response	Immutable audit log, anomaly detection	Breaches detected and traced

1.3 Zero-Trust Principles

Helix OTA adopts zero-trust networking principles as defined by NIST SP 800-207. The core tenets applied are:

1. **Never trust, always verify.** Every API request is authenticated and authorized regardless of source network. There is no “internal trusted network” — the device API and the dashboard API enforce identical authentication rigor.

2. **Least-privilege access.** Device certificates grant access only to device-facing endpoints. Dashboard users receive only the permissions their role requires. API keys are scoped to specific operations.

3. **Assume breach.** The system is designed to limit blast radius if any single component is compromised. A compromised device certificate cannot access the admin API. A stolen admin JWT cannot modify artifacts without also possessing the signing key stored in the HSM.

4. **Explicit verification at every step.** The artifact validation pipeline does not trust the upload source — it re-verifies hash, signature, structure, and compatibility from scratch. The device does not trust the server’s download URL — it independently verifies the SHA-256 hash and RSA-4096 signature of the downloaded payload before handing it to `update_engine`.

5. **Continuous monitoring.** Every authentication event, every artifact upload, every rollout change, and every device status transition is logged to an immutable audit trail. Anomaly detection runs continuously against telemetry streams.

---

2. Threat Model (STRIDE Analysis)

The following STRIDE analysis identifies threats specific to the OTA update domain. Each threat is classified by category, assessed for impact and likelihood, and mapped to specific controls.

2.1 Spoofing

#	Threat	Impact	Likelihood	Controls
S1	**Unauthorized device registers as legitimate device** — Attacker provisions a rogue device with a forged device ID and attempts to register it with the OTA server to receive firmware or inject telemetry.	HIGH	MEDIUM	mTLS with CA-issued device certificates; certificate CN must match declared device_id; hardware fingerprint binding; device registration rate limiting (5 req/min per cert CN)
S2	**Forged update source** — Attacker spoofs the OTA server URL and attempts to serve malicious payloads to devices.	CRITICAL	LOW	TLS 1.3 with certificate pinning on the client; server certificate CN validated against pinned `*.helix-ota.io`; RSA-4096 artifact signature verified on device (independent of transport)
S3	**Dashboard session hijacking** — Attacker steals	HIGH	MEDIUM	Short-lived access tokens (15 min); TOTP 2FA for privileged

#	Threat	Impact	Likelihood	Controls
	a valid JWT and uses it to perform admin operations.			roles; IP binding on JWT claims; refresh token rotation with reuse detection; HTTP-only secure cookies
S4	**API key theft** — An automation API key is leaked and used to authenticate as a service.	MEDIUM	MEDIUM	API keys are scoped to specific operations; keys have configurable expiration; key usage is logged to audit trail; keys can be instantly revoked

2.2 Tampering

#	Threat	Impact	Likelihood	Controls
T1	**Modified OTA payload in transit** — MITM attacker intercepts and modifies the artifact binary during download.	CRITICAL	LOW	TLS 1.3 prevents in-transit modification; SHA-256 hash verified on device against server-provided hash; RSA-4096 signature verified on device against embedded public key — even if TLS is broken, the signature check catches tampering
T2	**Modified OTA payload at rest on storage** — Attacker gains access to S3/MinIO and replaces an artifact with a malicious one.	CRITICAL	MEDIUM	RSA-4096 signature stored separately from artifact; signature verified on download; S3 bucket policies enforce write-once for validated artifacts; artifact hash in database is immutable after validation;

#	Threat	Impact	Likelihood	Controls
				S3 server-side encryption (SSE-S3)
T3	**Modified artifact metadata** — Attacker alters the `target_version` or `hardware_compatibility` fields in the database to target the wrong devices.	HIGH	LOW	Database row-level security; RBAC restricts artifact metadata modification to admin role; all metadata changes are logged to audit_log with before/after values; `artifacts` table columns are immutable after `upload_status = 'ready'` (enforced by application layer)
T4	**Tampered build pipeline output** — The CI/CD pipeline that produces OTA artifacts is compromised, producing signed but malicious builds.	CRITICAL	LOW	Signing keys stored in HSM; signing operation requires dual-operator approval (future: 1.1.0); build pipeline runs in isolated, hardened containers; reproducible builds verified against independent build (future)

2.3 Repudiation

#	Threat	Impact	Likelihood	Controls
R1	**Denying update installation** — A device claims it never received or installed an update that	MEDIUM	LOW	Complete audit trail: rollout creation → device update record → telemetry events (download_start, install_complete, commit) → device version

#	Threat	Impact	Likelihood	Controls
	was actually deployed.			update. All events are timestamped and signed with device certificate.
R2	**Audit trail gaps** — An attacker with database access deletes audit_log entries to cover their tracks.	HIGH	MEDIUM	audit_log is append-only (no UPDATE or DELETE permissions for application role); database triggers prevent deletion; write-ahead log (WAL) archiving provides point-in-time recovery; periodic export to immutable object storage (S3 Object Lock)
R3	**Operator denies performing an action** — An admin claims they did not pause a rollout or delete an artifact.	MEDIUM	LOW	Every dashboard action is logged to audit_log with user_id, IP address, and timestamp; JWT token ID is included in audit records; session cannot be shared (concurrent session detection)

2.4 Information Disclosure

#	Threat	Impact	Likelihood	Controls
I1	**Leaked device credentials** — A device's mTLS private key is extracted from the filesystem.	HIGH	MEDIUM	Private keys stored in Android Keystore (hardware-backed on RK3588); keys are non-exportable; certificate revocation via CRL/OCSP; device deprovisioning invalidates the certificate

#	Threat	Impact	Likelihood	Controls
I2	**Exposed device data via API** — An attacker queries the device API and extracts the full device inventory including serial numbers, IP addresses, and firmware versions.	HIGH	MEDIUM	Device listing API requires admin/operator JWT; device self-registration only returns the registering device's own data; field-level filtering (IP addresses only visible to admin); rate limiting prevents bulk enumeration
I3	**Database breach** — Attacker gains read access to PostgreSQL and extracts device records, user password hashes, and TOTP secrets.	CRITICAL	LOW	Database encryption at rest (PostgreSQL TDE or LUKS); network-isolated database (no public IP); TLS for all database connections; bcrypt with cost 12 for password hashes; TOTP secrets encrypted with AES-256-GCM before storage; database credentials rotated quarterly
I4	**Leaked signing key** — The RSA-4096 artifact signing private key is extracted from the server or HSM.	CRITICAL	VERY LOW	Signing key stored in HSM (HashiCorp Vault Transit or AWS KMS); key never leaves HSM boundary; signing operation is a remote API call to the HSM; key access requires dual-authentication; key rotation procedure (see §4.5)

2.5 Denial of Service

#	Threat	Impact	Likelihood	Controls
D1	**Update server overwhelm** — Attacker floods the OTA API with requests, preventing legitimate devices from checking for updates.	HIGH	MEDIUM	Rate limiting per device (60 req/min) and per IP (global); Redis-backed token bucket with distributed coordination; auto-scaling of server replicas; circuit breaker on downstream dependencies
D2	**Device bricking via bad update** — A corrupted or incompatible OTA package is deployed, rendering devices unbootable.	CRITICAL	LOW	Four-stage artifact validation; A/B partition update with automatic rollback; dm-verity and AVB boot verification; phased rollout with health-gated auto-advance (5% failure rate threshold); manual rollback capability
D3	**Storage exhaustion** — Attacker uploads large artifact files to exhaust S3 storage quota.	MEDIUM	LOW	2 GB max artifact size enforced at API gateway and application layer; artifact upload rate limit (10/hour per user); upload requires admin/operator JWT; orphaned artifacts cleaned by scheduled maintenance job
D4	**Database connection pool exhaustion** — Sustained query load depletes the pgx	MEDIUM	MEDIUM	Connection pool limits (50 max conns); query timeout enforcement; read replicas for dashboard queries; Redis

#	Threat	Impact	Likelihood	Controls
	connection pool.			cache for frequent queries (60s TTL)

2.6 Elevation of Privilege

#	Threat	Impact	Likelihood	Controls
E1	**Compromised device gains server access** — A rooted device uses its mTLS certificate to access admin-only endpoints.	HIGH	LOW	Device certificates have OU=device; middleware validates OU claim and routes to device-only API surface; device API has no admin operations; device tokens contain role=device claim; RBAC enforces device cannot assume admin/operator/ viewer roles
E2	**Rogue admin** — A user with admin role creates a backdoor account or exfiltrates signing keys.	CRITICAL	LOW	Admin actions logged to immutable audit_log; TOTP 2FA required for admin role; signing key operations require separate HSM authentication (not just dashboard JWT); critical operations (artifact deletion, user creation) require re-authentication; break-glass procedures with incident ticket requirement
E3	**SQL injection** — Attacker	CRITICAL	LOW	Parameterized queries

#	Threat	Impact	Likelihood	Controls
	crafts input that executes arbitrary SQL on the database.			exclusively via pgx; no string concatenation for SQL; input validation with allow-lists (not deny-lists); ORM-style repository layer abstracts all query construction; automated SQL injection testing in CI pipeline
E4	**Container escape** — Attacker exploits a container vulnerability to gain host access.	CRITICAL	VERY LOW	Containers run as non-root with read-only filesystem; seccomp profile restricts syscalls; no privileged containers; minimal base image (distroless); regular image scanning for CVEs; Kubernetes network policies isolate pods

3. Transport Security

3.1 TLS 1.3 — Mandatory for All Connections

TLS 1.3 is mandatory for every connection to the Helix OTA server. TLS 1.2 is not supported. This eliminates downgrade attacks and removes obsolete cipher suites from the attack surface.

Approved Cipher Suites (in preference order):

Cipher Suite	Key Exchange	Auth	Cipher	Hash
`TLS_AES_256_GCM_SHA384`	ECDHE	RSA/ ECDSA	AES-256-GCM	SHA-384
`TLS_CHACHA20_POLY1305_SHA256`	ECDHE			SHA-256

Cipher Suite	Key Exchange	Auth	Cipher	Hash
		RSA/ ECDSA	ChaCha20-Poly1305	
TLS_AES_128_GCM_SHA256	ECDHE	RSA/ ECDSA	AES-128-GCM	SHA-256

Go TLS Configuration:

```go
package tlsconfig

import (
    "crypto/tls"
    "crypto/x509"
    "os"
)

// ServerTLSConfig returns a hardened TLS 1.3 configuration for
        the OTA server.
func ServerTLSConfig(caCertPath string) (*tls.Config, error) {
    caCert, err := os.ReadFile(caCertPath)
    if err != nil {
        return nil, fmt.Errorf("read CA cert: %w", err)
    }

    caPool := x509.NewCertPool()
    if !caPool.AppendCertsFromPEM(caCert) {
        return nil, fmt.Errorf("failed to append CA cert")
    }

    return &tls.Config{
        MinVersion: tls.VersionTLS13,
        MaxVersion: tls.VersionTLS13,

        // Only allow AEAD cipher suites (TLS 1.3 requirement)
        CipherSuites: []uint16{
            tls.TLS_AES_256_GCM_SHA384,
            tls.TLS_CHACHA20_POLY1305_SHA256,
            tls.TLS_AES_128_GCM_SHA256,
        },

        // ECDHE key exchange with P-256 minimum (P-384 preferred)
        CurvePreferences: []tls.CurveID{
            tls.X25519,
            tls.CurveP384,
            tls.CurveP256,
        },

        // mTLS: require client certificate for device endpoints
        ClientAuth: tls.VerifyClientCertIfGiven,
        ClientCAs:  caPool,
```

```
        // Prevent session resumption across compromised
        connections
        SessionTicketsDisabled: true,

        // Strict certificate verification
        InsecureSkipVerify: false,
    }, nil
}
```

3.2 Certificate Pinning on Client

The Android client pins the expected server certificate or CA to prevent MITM attacks even if a trusted CA is compromised. Pinning is implemented at two levels:

1. **CA Pin:** The root CA that issues the server certificate is pinned. This allows certificate rotation within the same CA without client updates.
2. **Backup Pin:** A secondary CA pin is included for CA migration scenarios.

```
package client

import (
    "crypto/sha256"
    "crypto/tls"
    "crypto/x509"
    "encoding/hex"
    "errors"
)

// PinnedCerts contains the SHA-256 hashes of trusted CA public
        keys.
// These are compiled into the client binary and cannot be
        remotely modified.
var PinnedCerts = struct {
    Primary   string // SHA-256 of primary CA public key
    Secondary string // SHA-256 of backup CA public key (for
        rotation)
}{
    Primary:
        "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2",
    Secondary:
        "c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4",
}

// NewPinnedTLSConfig creates a TLS config that validates the
        server's
// certificate chain against pinned CA public key hashes.
func NewPinnedTLSConfig() *tls.Config {
    return &tls.Config{
        MinVersion: tls.VersionTLS13,
        MaxVersion: tls.VersionTLS13,
        VerifyConnection: func(cs tls.ConnectionState) error {
            for _, cert := range cs.PeerCertificates {
                pubKeyHash :=
        sha256.Sum256(cert.RawSubjectPublicKeyInfo)
```

```
                hashHex := hex.EncodeToString(pubKeyHash[:])

                if hashHex == PinnedCerts.Primary || hashHex ==
        PinnedCerts.Secondary {
                    return nil // Pin matched
                }
            }
            return errors.New("server certificate does not match
        any pinned CA")
        },
    }
}
```

3.3 mTLS for Device-to-Server Authentication

All device-facing endpoints require mutual TLS. The device presents its client certificate during the TLS handshake, and the server validates it against the Helix OTA Device CA. This provides:

- **Cryptographic device identity** — The private key never leaves the device's Android Keystore.
- **No credential in request body** — Unlike API keys or JWTs, mTLS credentials cannot be intercepted via request logging.
- **Replay protection** — TLS handshake is fresh per connection; certificates cannot be replayed.

```
package middleware

import (
    "crypto/x509"
    "net/http"
    "strings"
)

// DeviceMTLSMiddleware extracts the device identity from the mTLS
        client
// certificate and injects it into the request context.
func DeviceMTLSMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
        if r.TLS == nil {
            http.Error(w, "TLS required",
        http.StatusUpgradeRequired)
            return
        }

        certs := r.TLS.PeerCertificates
        if len(certs) == 0 {
            writeAPIError(w, http.StatusUnauthorized,
        "MTLS_REQUIRED",

        "No client certificate presented. Device endpoints require
        mTLS.")
            return
```

```
        }

        cert := certs[0]

        // Validate certificate OU (Organizational Unit) is
        "device"
        var isDevice bool
        for _, ou := range cert.Subject.OrganizationalUnit {
            if ou == "device" {
                isDevice = true
                break
            }
        }
        if !isDevice {
            writeAPIError(w, http.StatusForbidden,
        "INVALID_CERT_ROLE",
                "Client certificate is not authorized for device
        endpoints.")
            return
        }

        // Extract device ID from CN
        deviceID := cert.Subject.CommonName
        if deviceID == "" || !isValidDeviceID(deviceID) {
            writeAPIError(w, http.StatusForbidden,
        "INVALID_DEVICE_ID",
                "Certificate CN does not contain a valid device
        identifier.")
            return
        }

        // Verify certificate is not revoked (CRL/OCSP check)
        if isRevoked(cert) {
            writeAPIError(w, http.StatusUnauthorized,
        "CERT_REVOKED",
                "Device certificate has been revoked.")
            return
        }

        // Inject device identity into context
        ctx := contextWithDeviceID(r.Context(), deviceID)
        ctx = contextWithAuthMethod(ctx, "mTLS")
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```

3.4 Certificate Rotation Strategy

Certificate Type	Validity	Rotation Procedure	Overlap Period
Server TLS Certificate	90 days	Automated via ACME (Let's	

Certificate Type	Validity	Rotation Procedure	Overlap Period
		Encrypt) or internal CA; zero-downtime via cert hot-reload	30 days (new cert issued 30 days before expiry)
Device Client Certificate	1 year	Server pushes renewal notification via check-in response; device re-enrolls using existing valid cert; new cert issued with overlapping validity	60 days (device can use old cert until expiry)
Device CA Certificate	10 years	New CA cert distributed via firmware update; server trusts both old and new CA during transition	2 years (old CA remains in trust store until all devices have new CA)
Signing Key	2 years	See §4.5 Key Rotation Procedure	90 days (devices trust both old and new public keys)

4. Artifact Security

Artifact security is the most critical layer in the Helix OTA architecture. A single compromised artifact can affect every device in the fleet. The following controls ensure that no artifact reaches a device unless it has been cryptographically verified at multiple independent points.

4.1 RSA-4096 Signing of OTA Artifacts

Every OTA artifact is signed with an RSA-4096 private key stored exclusively in an HSM. The signature covers the entire `payload.bin` file within the OTA ZIP archive. RSA-4096 was chosen over ECDSA for its broader platform support (Android `update_engine` has native RSA verification) and its resistance to future quantum computing attacks (requiring approximately 2^{80} operations to break with Grover's algorithm, compared to ~2^{64} for ECDSA P-256).

```
package signing

import (
    "crypto"
    "crypto/rand"
    "crypto/rsa"
    "crypto/sha256"
```

```
    "fmt"
    "io"
    "os"
)

// ArtifactSigner signs OTA artifacts using an RSA-4096 private
        key.
// In production, this delegates to HashiCorp Vault's Transit
        engine
// so the private key never leaves the HSM.
type ArtifactSigner struct {
    // signer is either a local *rsa.PrivateKey (dev) or a Vault
        Transit client (prod)
    signer RSASigner
}

// RSASigner abstracts the signing operation for testability and
        HSM delegation.
type RSASigner interface {
    Sign(rand io.Reader, digest []byte, opts crypto.SignerOpts)
        ([]byte, error)
}

// SignArtifact computes the SHA-256 hash of the artifact file and
        signs it.
func (s *ArtifactSigner) SignArtifact(ctx context.Context,
        artifactPath string) ([]byte, error) {
    // 1. Stream-read the artifact and compute SHA-256
    f, err := os.Open(artifactPath)
    if err != nil {
        return nil, fmt.Errorf("open artifact: %w", err)
    }
    defer f.Close()

    hash := sha256.New()
    if _, err := io.Copy(hash, f); err != nil {
        return nil, fmt.Errorf("compute hash: %w", err)
    }
    digest := hash.Sum(nil)

    // 2. Sign the digest with RSA-4096 + PSS padding
    signature, err := s.signer.Sign(rand.Reader, digest,
        &rsa.PSSOptions{
        SaltLength: rsa.PSSSaltLengthEqualsHash,
        Hash:       crypto.SHA256,
    })
    if err != nil {
        return nil, fmt.Errorf("sign artifact: %w", err)
    }

    return signature, nil
}
```

4.2 Signature Verification Chain on Device

The device performs independent signature verification using a public key embedded in the read-only firmware partition. This public key is provisioned during manufacturing and cannot be modified without a signed firmware update. The verification chain is:

```
payload.bin → SHA-256 digest → RSA-4096-PSS verification with
embedded public key → VALID/INVALID
```

```
package verification

import (
    "crypto"
    "crypto/rsa"
    "crypto/sha256"
    "encoding/hex"
    "errors"
    "fmt"
    "io"
    "os"
)

// EmbeddedPublicKey is the RSA-4096 public key compiled into the
        device firmware.
// This key is stored in the read-only /system partition and
        protected by dm-verity.
// var EmbeddedPublicKey *rsa.PublicKey // initialized from
        embedded PEM

// VerifyArtifact performs the full client-side verification
        chain:
//   1. Compute SHA-256 of the downloaded artifact
//   2. Compare against the expected hash from the server
//   3. Verify the RSA-4096-PSS signature against the embedded
        public key
func VerifyArtifact(
    artifactPath string,
    expectedSHA256 string,
    signature []byte,
    publicKey *rsa.PublicKey,
) error {
    // Step 1: Compute SHA-256 of the downloaded file
    f, err := os.Open(artifactPath)
    if err != nil {
        return fmt.Errorf("open artifact: %w", err)
    }
    defer f.Close()

    hash := sha256.New()
    if _, err := io.Copy(hash, f); err != nil {
        return fmt.Errorf("compute hash: %w", err)
    }
    digest := hash.Sum(nil)
```

```
    computedHash := hex.EncodeToString(digest)

    // Step 2: Compare against server-provided hash
    if computedHash != expectedSHA256 {
        return
        fmt.Errorf("SHA-256 mismatch: expected %s, computed %s",
            expectedSHA256, computedHash)
    }

    // Step 3: Verify RSA-4096-PSS signature
    err = rsa.VerifyPSS(publicKey, crypto.SHA256, digest,
        signature, &rsa.PSSOptions{
        SaltLength: rsa.PSSSaltLengthEqualsHash,
        Hash:       crypto.SHA256,
    })
    if err != nil {
        return fmt.Errorf("RSA signature verification failed:
        %w", err)
    }

    return nil
}
```

4.3 SHA-256 Hash Verification

Hash verification serves as the first and fastest integrity check. The build pipeline produces a `.sha256` file alongside each artifact. The server independently computes the hash during upload and stores it immutably in the `artifacts.file_hash_sha256` column (validated by CHECK constraint `^[a-f0-9]{64}$`).

Upload Validation:

```
package validation

import (
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "io"
)

// HashValidator computes and verifies SHA-256 hashes during
        upload.
type HashValidator struct{}

// ValidateStreamToTemp reads from the input stream, writes to a
        temp file,
// and computes the SHA-256 hash simultaneously. Returns the temp
        file path,
// the hex-encoded hash, and the total bytes written.
func (v *HashValidator) ValidateStreamToTemp(
    reader io.Reader,
```

```
    expectedHash string,
    maxBytes int64,
) (tempPath string, computedHash string, size int64, err error) {
    tmpFile, err := os.CreateTemp("", "helix-upload-*.tmp")
    if err != nil {
        return "", "", 0, fmt.Errorf("create temp file: %w", err)
    }
    tempPath = tmpFile.Name()

    defer tmpFile.Close()

    hash := sha256.New()
    writer := io.MultiWriter(tmpFile, hash)

    // LimitReader prevents storage exhaustion attacks
    limitedReader := io.LimitReader(reader, maxBytes)
    size, err = io.Copy(writer, limitedReader)
    if err != nil {
        os.Remove(tempPath)
        return "", "", 0, fmt.Errorf("write to temp: %w", err)
    }

    digest := hash.Sum(nil)
    computedHash = hex.EncodeToString(digest)

    // If an expected hash was provided, verify it
    if expectedHash != "" && computedHash != expectedHash {
        os.Remove(tempPath)
        return "", "", 0, fmt.Errorf("hash mismatch: expected %s,
        got %s",
            expectedHash, computedHash)
    }

    return tempPath, computedHash, size, nil
}
```

4.4 Upload Validation Pipeline

The upload validation pipeline is a four-stage chain. All four stages must pass for the artifact to transition to `upload_status = 'ready'`. The pipeline is implemented as a sequential chain within a worker pool, with each stage's result recorded in the `artifact_validation_results` table.

```
Stage 1: Hash Verification      ──►  PASS/FAIL
Stage 2: Signature Verification ──►  PASS/FAIL
Stage 3: Structure Check        ──►  PASS/FAIL
Stage 4: Compatibility Check    ──►  PASS/FAIL
         │
         ▼
  All PASS → upload_status = 'ready'
  Any FAIL → upload_status = 'failed'
```

```
package validation

import (
    "archive/zip"
    "context"
    "fmt"
    "strings"
)

// ValidationChain executes the four-stage artifact validation
        pipeline.
type ValidationChain struct {
    hashChecker         HashChecker
    signatureChecker    SignatureChecker
    structureChecker    StructureChecker
    compatibilityChecker CompatibilityChecker
}

// Validate runs all four validation stages and returns the
        combined result.
func (vc *ValidationChain) Validate(
    ctx context.Context,
    artifactPath string,
    artifact *Artifact,
) (*ValidationResult, error) {
    result := &ValidationResult{}

    // Stage 1: Hash verification
    stage1 := vc.hashChecker.Check(ctx, artifactPath, artifact)
    result.Stages = append(result.Stages, stage1)
    if !stage1.Passed {
        result.Valid = false
        result.Errors = append(result.Errors, stage1.Message)
        return result, nil // Short-circuit: no point checking
        signature of corrupted file
    }

    // Stage 2: Signature verification
    stage2 := vc.signatureChecker.Check(ctx, artifactPath,
        artifact)
    result.Stages = append(result.Stages, stage2)
    if !stage2.Passed {
        result.Valid = false
        result.Errors = append(result.Errors, stage2.Message)
        return result, nil // Short-circuit: unsigned artifact
        cannot be trusted
    }

    // Stage 3: Structure check (ZIP must contain required files)
    stage3 := vc.structureChecker.Check(ctx, artifactPath,
        artifact)
    result.Stages = append(result.Stages, stage3)
    if !stage3.Passed {
        result.Valid = false
```

```
        result.Errors = append(result.Errors, stage3.Message)
        return result, nil
    }

    // Stage 4: Compatibility check (target model + version
        constraints)
    stage4 := vc.compatibilityChecker.Check(ctx, artifactPath,
        artifact)
    result.Stages = append(result.Stages, stage4)
    if !stage4.Passed {
        result.Valid = false
        result.Errors = append(result.Errors, stage4.Message)
        return result, nil
    }

    result.Valid = true
    return result, nil
}

// StructureChecker validates that the OTA ZIP contains the
        required files
// for Android 15 A/B update on RK3588.
type StructureChecker struct{}

func (sc *StructureChecker) Check(ctx context.Context, path
        string, a *Artifact) ValidationStage {
    zr, err := zip.OpenReader(path)
    if err != nil {
        return ValidationStage{Name: "structure", Passed: false,
            Message: fmt.Sprintf("cannot open ZIP: %v", err)}
    }
    defer zr.Close()

    requiredFiles := []string{
        "payload.bin",             // Main update payload
        "payload_properties.txt",  // Payload metadata
        "care_map.pb",             // dm-verity care map for post-
        install verification
    }

    found := make(map[string]bool)
    for _, f := range zr.File {
        found[f.Name] = true
    }

    var missing []string
    for _, req := range requiredFiles {
        if !found[req] {
            missing = append(missing, req)
        }
    }

    if len(missing) > 0 {
```

```
            return ValidationStage{Name: "structure", Passed: false,
                Message: fmt.Sprintf("missing required files: %s",
                strings.Join(missing, ", "))}
        }

        return ValidationStage{Name: "structure", Passed: true,
            Message: "OTA package structure is valid"}
}
```

4.5 Key Management and Rotation

Key Storage: The RSA-4096 signing private key is stored exclusively in a Hardware Security Module (HSM). In production, this is HashiCorp Vault's Transit secrets engine or AWS KMS. The key never exists in plaintext outside the HSM boundary. Signing is a remote API call: the server sends the hash digest to the HSM, and the HSM returns the signature.

Key Hierarchy:

```
Root Signing Key (RSA-4096, HSM-stored, 10-year validity)
  └── Artifact Signing Key (RSA-4096, HSM-stored, 2-year validity)
       └── Device-embedded Public Key (matching Artifact Signing
Key)
```

Key Rotation Procedure:

1. **Generate new key pair** inside the HSM. The new key is registered as a secondary signing key.
2. **Update server configuration** to sign new artifacts with the new key while still accepting artifacts signed with the old key.
3. **Deploy firmware update** containing the new public key alongside the old public key. Both keys are trusted during the overlap period.
4. **Wait for fleet convergence** — all devices must be running firmware that trusts the new key before the old key can be retired.
5. **Re-sign existing active artifacts** with the new key (server-side re-signing from storage).
6. **Revoke old key** in HSM and remove from server configuration.
7. **Remove old public key** from the next firmware release after fleet convergence is confirmed.

Total rotation window: 90 days (allows for phased rollout of the new public key firmware).

---

5. Authentication & Authorization

5.1 JWT Token Management

The dashboard uses a dual-token JWT strategy: short-lived access tokens for API calls and longer-lived refresh tokens for obtaining new access tokens.

Token Type	Lifetime	Storage	Rotation
Access Token	15 minutes	Memory (JavaScript variable)	Automatic via refresh token
Refresh Token	7 days	HTTP-only, Secure, SameSite=Strict cookie	Single-use rotation on each refresh

JWT Claims:

```
type AccessTokenClaims struct {
    jwt.RegisteredClaims

    // Custom claims
    UserID   string `json:"uid"`  // User UUID
    Role     string `json:"role"` // admin, operator, viewer
    SessionID string `json:"sid"` // Unique session identifier
    TokenType string `json:"typ"` // "access"
}

type RefreshTokenClaims struct {
    jwt.RegisteredClaims

    UserID    string `json:"uid"`
    SessionID string `json:"sid"`
    TokenType string `json:"typ"` // "refresh"
    TokenFamily string `json:"fam"` // Token family for reuse
        detection
}
```

Refresh Token Rotation with Reuse Detection:

When a refresh token is used, it is immediately invalidated and a new refresh token is issued. If a previously-used refresh token is presented (indicating potential theft), the server invalidates **all** tokens in the same token family, forcing the user to re-authenticate:

```
package auth

import (
    "errors"
    "time"
)

var ErrTokenReuseDetected = errors.New("refresh token reuse
        detected — all sessions invalidated")

// RefreshAccessToken handles the refresh token rotation flow.
// If a previously-used refresh token is presented, all sessions
        for the
// user are invalidated as a theft countermeasure.
func (s *AuthService) RefreshAccessToken(
```

```
    ctx context.Context,
    oldRefreshToken string,
) (*TokenPair, error) {
    // 1. Parse and validate the refresh token
    claims, err := s.jwtProvider.ValidateRefreshToken(ctx,
        oldRefreshToken)
    if err != nil {
        return nil, ErrInvalidRefreshToken
    }

    // 2. Check if this refresh token has already been used
    used, err := s.repo.IsRefreshTokenUsed(ctx, claims.ID)
    if err != nil {
        return nil, fmt.Errorf("check token usage: %w", err)
    }
    if used {
        // TOKEN REUSE DETECTED — potential theft
        // Invalidate ALL sessions for this user
        s.invalidateAllUserSessions(ctx, claims.UserID)

        // Log security event
        s.events.Publish(ctx, eventbus.Event{
            Type: "security.token_reuse_detected",
            Payload: map[string]interface{}{
                "user_id":    claims.UserID,
                "session_id": claims.SessionID,
                "token_jti":  claims.ID,
            },
        })

        return nil, ErrTokenReuseDetected
    }

    // 3. Mark the old refresh token as used
    if err := s.repo.MarkRefreshTokenUsed(ctx, claims.ID,
        time.Now()); err != nil {
        return nil, fmt.Errorf("mark token used: %w", err)
    }

    // 4. Issue new access + refresh token pair
    newPair, err := s.issueTokenPair(ctx, claims.UserID,
        claims.Role, claims.SessionID)
    if err != nil {
        return nil, fmt.Errorf("issue tokens: %w", err)
    }

    return newPair, nil
}
```

5.2 TOTP 2FA for Dashboard Users

All admin and operator accounts require TOTP two-factor authentication. Viewer accounts may optionally enable TOTP. The TOTP implementation follows RFC 6238 with the following parameters:

- **Algorithm:** HMAC-SHA-1 (for compatibility with standard authenticator apps)
- **Time Step:** 30 seconds
- **Code Length:** 6 digits
- **Skew Window:** 1 time step before and after (to account for clock drift)

Security Hardening: - Backup codes (6 codes, 8 characters each, single-use) are generated during TOTP setup - Failed TOTP attempts are rate-limited: 5 failures per 15 minutes, then account lockout for 30 minutes - TOTP secret is encrypted with AES-256-GCM before storage in the `users.totp_secret` column - The encryption key for TOTP secrets is stored in Vault, not in environment variables

5.3 Device mTLS Certificate Provisioning

Device certificates follow a specific provisioning workflow:

1. **Manufacturing Phase:** Each device is provisioned with a unique RSA-3072 key pair generated inside the Android Keystore (hardware-backed on RK3588). The public key is sent to the Helix OTA Certificate Authority.
2. **CA Signing:** The Helix OTA Device CA (intermediate CA) signs the device's CSR, producing a certificate with:
   - `CN = {device_id}.helix-ota.io`
   - `OU = device`
   - Validity: 1 year
   - Key Usage: Digital Signature, Key Encipherment
   - Extended Key Usage: TLS Client Authentication
3. **Certificate Installation:** The signed certificate is installed in the device's Android Keystore and is non-exportable.
4. **Renewal:** 60 days before expiry, the server includes a `certificate_renewal_required` flag in the update check response. The device re-enrolls using its existing valid certificate.

5.4 RBAC Matrix

Resource / Operation	Admin	Operator	Viewer	Device
Artifacts				
Upload artifact	✅	✅	❌	❌
View artifact list	✅	✅	✅	❌
Download artifact	✅	✅	❌	✅
Delete artifact	✅	❌	❌	❌
Re-validate artifact	✅	✅	❌	❌
Rollouts				
Create rollout	✅	✅	❌	❌
View rollout	✅	✅	✅	❌

Resource / Operation	Admin	Operator	Viewer	Device
Pause/resume rollout	✅	✅	❌	❌
Advance rollout	✅	✅	❌	❌
Halt/rollback rollout	✅	❌	❌	❌
Devices				
Register device (mTLS)	—	—	—	✅
View device list	✅	✅	✅	❌
Update device metadata	✅	✅	❌	❌
Decommission device	✅	❌	❌	❌
Trigger rollback	✅	✅	❌	❌
Check for updates	—	—	—	✅
Report telemetry	—	—	—	✅
Users				
Create user	✅	❌	❌	❌
Update user role	✅	❌	❌	❌
Delete user	✅	❌	❌	❌
View user list	✅	❌	❌	❌
Telemetry				
View telemetry	✅	✅	✅	❌
Export telemetry	✅	✅	❌	❌

5.5 API Key Management

API keys are used for service-to-service automation (e.g., CI/CD pipelines uploading artifacts). API keys:

- Are prefixed with `hota_` for identification
- Are hashed (SHA-256) before storage — the plaintext is shown only once at creation
- Have configurable scopes (e.g., `artifact:upload`, `rollout:read`)
- Have configurable expiration (default: 90 days, max: 365 days)
- Are rate-limited independently from user-based rate limits
- Are logged in the audit trail on every use

```
package auth

import (
    "crypto/rand"
    "crypto/sha256"
    "encoding/hex"
    "fmt"
)

const apiKeyPrefix = "hota_"

// GenerateAPIKey creates a new API key and returns the plaintext
        (shown once)
```

```
// along with the hash for storage.
func GenerateAPIKey() (plaintext string, hash string, err error) {
    bytes := make([]byte, 32)
    if _, err := rand.Read(bytes); err != nil {
        return "", "", fmt.Errorf("generate random bytes: %w",
        err)
    }

    plaintext = apiKeyPrefix + hex.EncodeToString(bytes)
    hashBytes := sha256.Sum256([]byte(plaintext))
    hash = hex.EncodeToString(hashBytes[:])

    return plaintext, hash, nil
}
```

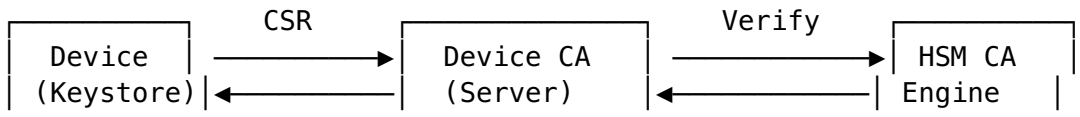
5.6 Session Management

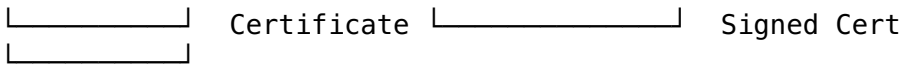
Property	Value	Rationale
Maximum concurrent sessions per user	3	Prevents credential sharing; oldest session evicted
Session idle timeout	30 minutes	Dashboard auto-logout after inactivity
Session absolute timeout	8 hours	Forces re-authentication even if active
Session invalidation on password change	All sessions	Prevents continued access after credential rotation
Session invalidation on role change	All sessions	Ensures permission changes take effect immediately

6. Device Identity

6.1 Device Provisioning and Certificate Enrollment

Device provisioning is a multi-step process that binds a cryptographic identity to a physical device:





1. **Key Generation:** The device generates an RSA-3072 key pair inside the Android Keystore. On RK3588, the Keystore is backed by the Trusted Execution Environment (TEE), making the private key hardware-protected and non-exportable.

2. **CSR Submission:** The device creates a Certificate Signing Request (CSR) containing the public key and the device ID (derived from hardware serial number). The CSR is signed with the device's private key.

3. **Server Validation:** The server validates:

   - The CSR signature is valid (proves possession of the private key)
   - The device ID does not already exist in the database
   - The device ID matches the hardware fingerprint provided in the registration request

4. **Certificate Issuance:** The Device CA (backed by the HSM) signs the certificate with the standard device certificate profile.

6.2 Device ID Generation

The device ID is hardware-bound and deterministic. It is derived from multiple hardware identifiers to prevent cloning:

```go
package device

import (
    "crypto/sha256"
    "encoding/hex"
    "fmt"
)

// DeviceIDInput contains the hardware identifiers used to derive
    a device ID.
type DeviceIDInput struct {
    SerialNumber    string // CPU serial number from /sys/class/
        serial/serial_number
    MACAddress      string // Primary network interface MAC
        address
    RPMSerial       string // RPM serial from /sys/class/rpmsg/
        rpmsg0/device/serial
    BoardRevision   string // PCB revision from /proc/device-tree/
        revision
}

// GenerateDeviceID produces a deterministic, hardware-bound
    device identifier.
// The ID is: "dev_" + hex(SHA-256(serial + mac + rpm_serial +
    board_revision))
// Using multiple identifiers prevents cloning: an attacker would
    need to forge
// all four hardware attributes to impersonate a device.
```

```
func GenerateDeviceID(input DeviceIDInput) string {
    preimage := fmt.Sprintf("%s|%s|%s|%s",
        input.SerialNumber,
        input.MACAddress,
        input.RPMSerial,
        input.BoardRevision,
    )

    hash := sha256.Sum256([]byte(preimage))
    return "dev_" + hex.EncodeToString(hash[:16]) // 32 hex chars
        after prefix
}
```

6.3 Device Deprovisioning and Certificate Revocation

When a device is decommissioned (via `DELETE /api/v1/devices/{device_id}`):

1. The device record's `deleted_at` is set to the current timestamp (soft delete).
2. The device's mTLS certificate is added to the Certificate Revocation List (CRL).
3. An OCSP "revoked" status is recorded for the certificate serial number.
4. All pending device_updates for this device are cancelled.
5. The decommissioning is logged to `audit_log` with the admin's user_id, the reason, and the IP address.
6. The device is removed from all device group memberships.

CRL distribution points are configured in the Device CA certificate. The server's TLS configuration checks the CRL on every mTLS handshake, ensuring that a decommissioned device cannot authenticate even if its certificate has not yet expired.

6.4 Anti-Replay Protection

Replay attacks are mitigated at multiple levels:

- **TLS 1.3** provides implicit replay protection for the handshake via the `random` field in the ClientHello and ServerHello messages.
- **JWT tokens** include a `jti` (JWT ID) claim that is checked against a Redis-backed replay cache (TTL = token lifetime). A previously-used JWT is rejected.
- **Device telemetry events** include a monotonically increasing sequence number. The server rejects events with sequence numbers less than or equal to the last recorded sequence for the device.
- **Artifact download URLs** are signed with a time-limited HMAC (1-hour expiry). A downloaded URL cannot be reused after expiry.

```
package download

import (
    "crypto/hmac"
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "time"
```

```
)

// GenerateSignedDownloadURL creates a time-limited, tamper-proof
        download URL.
func GenerateSignedDownloadURL(
    baseURL string,
    artifactID string,
    secretKey []byte,
    expiresAt time.Time,
) string {
    expiryUnix := expiresAt.Unix()
    message := fmt.Sprintf("%s|%d", artifactID, expiryUnix)

    mac := hmac.New(sha256.New, secretKey)
    mac.Write([]byte(message))
    signature := hex.EncodeToString(mac.Sum(nil))

    return fmt.Sprintf("%s/api/v1/artifacts/%s/download?
        expires=%d&sig=%s",
        baseURL, artifactID, expiryUnix, signature)
}

// VerifySignedDownloadURL validates the HMAC signature and expiry
        of a download URL.
func VerifySignedDownloadURL(
    artifactID string,
    expiresUnix int64,
    signature string,
    secretKey []byte,
) error {
    // Check expiry
    if time.Now().Unix() > expiresUnix {
        return fmt.Errorf("download URL has expired")
    }

    // Verify HMAC
    message := fmt.Sprintf("%s|%d", artifactID, expiresUnix)
    mac := hmac.New(sha256.New, secretKey)
    mac.Write([]byte(message))
    expectedSig := hex.EncodeToString(mac.Sum(nil))

    if !hmac.Equal([]byte(signature), []byte(expectedSig)) {
        return fmt.Errorf("invalid download URL signature")
    }

    return nil
}
```

---

7. Update Safety Guarantees

7.1 Atomic Updates (A/B Partition Swap)

Helix OTA leverages Android's Virtual A/B update mechanism on the RK3588. The device maintains two complete sets of boot partitions:

Partition	Slot A	Slot B
Boot	`boot_a`	`boot_b`
System	`system_a`	`system_b`
Vendor	`vendor_a`	`vendor_b`
Product	`product_a`	`product_b`
ODM	`odm_a`	`odm_b`

The update process writes the new firmware to the **inactive** slot without touching the active slot. Only after verification passes does the Boot Control HAL mark the new slot as active. If the new slot fails to boot, the bootloader automatically falls back to the previous slot.

Key properties: - The update is **atomic**: the device either boots into the new version or stays on the old version. There is no intermediate state. - The active partition is **never modified** during the update, eliminating the risk of corruption from power failure during write. - The update is **non-blocking**: the device remains fully operational during the download and install phases.

7.2 Boot Verification (dm-verity, AVB)

dm-verity (Device Mapper Verity): The system and vendor partitions are protected by dm-verity, which verifies the integrity of every block as it is read. The dm-verity hash tree is stored in the `vbmeta` partition and is itself protected by Android Verified Boot (AVB).

AVB (Android Verified Boot): The `vbmeta` partition contains the hash descriptors for all verified partitions, signed by the OEM key. On boot, the bootloader:

1. Loads the `vbmeta` partition.
2. Verifies the AVB signature against the OEM public key (burned into eFuse during manufacturing).
3. Checks the hash descriptors against the actual partition data.
4. If verification fails, the boot is aborted and the device falls back to the previous slot.

Helix OTA Integration: The OTA artifact includes a `care_map.pb` file that contains the dm-verity care map for the new partitions. After installation, `update_engine` applies the care map so that dm-verity begins verifying the new partitions immediately on the next boot.

7.3 Automatic Rollback on Boot Failure

The RK3588 bootloader supports automatic rollback through the Android Boot Control HAL. The mechanism works as follows:

1. After writing the update to the inactive slot, `update_engine` marks the new slot as active with `boot_control.setActiveBootSlot(newSlot)`.
2. The new slot is marked with `unbootable = false` and `boot_successful = false`.
3. On reboot, the bootloader attempts to boot the new slot.
4. If the boot succeeds, the Android `BootCompletedReceiver` calls `boot_control.markBootSuccessful()`.
5. If the boot fails (kernel panic, dm-verity failure, or watchdog timeout), the bootloader:
   - Marks the slot as `unbootable = true`
   - Decrements the retry counter (default: 3 retries)
   - If retry count reaches 0, falls back to the previous slot
6. The device boots into the previous slot and reports a `rollback` telemetry event to the server.

7.4 Update Integrity Verification at Every Step

Integrity verification is performed at four distinct points in the update lifecycle:

Step	Verification	Implementation
1. Download complete	SHA-256 hash of downloaded file matches server-provided hash	Client SDK `DownloadManager` computes hash during download
2. Pre-install	RSA-4096 signature of `payload.bin` verifies against embedded public key	Client SDK `VerificationEngine` before invoking `update_engine`
3. Post-install (pre-boot)	`update_engine` verifies `payload.bin` internal integrity (delta hash tree)	Android `update_engine` built-in verification
4. Post-boot	dm-verity verifies every block of system/vendor partitions on read	Android dm-verity kernel module

7.5 Corrupted Update Detection and Rejection

A corrupted update is detected and rejected at the earliest possible point:

1. **During download:** If the SHA-256 hash of the downloaded file does not match the expected hash, the download is marked as failed and the file is deleted. The device reports a `HASH_MISMATCH` error via telemetry.

2. **During signature verification:** If the RSA-4096 signature does not verify, the artifact is rejected without being passed to `update_engine`. The device reports a `SIGNATURE_INVALID` error.

3. **During `update_engine` application:** If the `payload.bin` is internally inconsistent (corrupted delta blocks), `update_engine` aborts the installation and marks the inactive slot as unbootable. The active slot is unaffected.

4. **During post-install verification:** If the installed partition data does not match the dm-verity hash tree, the partition fails verification on first read and the boot is aborted with automatic rollback.

7.6 Power-Failure Safety During Update

The A/B update mechanism is inherently power-fail safe:

- **During download:** The partial download file is written to a temporary location. If power is lost, the partial file is detected on restart and the download resumes from the last byte (using HTTP Range headers). No partition data is modified.

- **During `update_engine` install:** The `update_engine` writes to the inactive slot using a COW (Copy-On-Write) snapshot for Virtual A/B. If power is lost during write:

  - The COW snapshot is in an incomplete state but the active slot is untouched.
  - On next boot, the bootloader detects the incomplete COW and discards it, falling back to the active slot.
  - The device reports the interrupted install and can retry.

- **During boot (post-install):** If power is lost during the first boot into the new slot, the boot retry counter ensures the bootloader attempts the new slot again up to 3 times before falling back.

---

8. Server Security

8.1 Container Security

The Helix OTA server runs as a containerized application with the following hardening measures:

```
# Multi-stage build: build stage has full toolchain, runtime is
        minimal
FROM golang:1.22-alpine AS builder
# ... build steps ...

# Runtime: distroless base image (no shell, no package manager, no
        attack surface)
FROM gcr.io/distroless/static-debian12:nonroot

# Copy only the compiled binary
COPY --from=builder /app/helix-ota-server /helix-ota-server

# Run as non-root user (UID 65534 = nobody)
USER 65534:65534
```

```
# Read-only root filesystem
# (Configured in Kubernetes:
        securityContext.readOnlyRootFilesystem: true)

# No privileged capabilities
# (Configured in Kubernetes: securityContext.drop: ["ALL"])

ENTRYPOINT ["/helix-ota-server"]
```

Kubernetes Security Context:

```
securityContext:
  runAsNonRoot: true
  runAsUser: 65534
  runAsGroup: 65534
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]
  seccompProfile:
    type: RuntimeDefault
```

Additional container hardening: - **No shell:** Distroless base image has no `/bin/sh`, preventing `kubectl exec` attacks - **No writable paths:** Temp files are written to an `emptyDir` volume mounted at `/tmp` - **Resource limits:** CPU and memory limits prevent resource exhaustion - **Network policy:** Egress restricted to PostgreSQL, Redis, MinIO/S3, and Vault only

8.2 Database Encryption at Rest

PostgreSQL data is encrypted at rest using one of the following methods (depending on deployment environment):

Environment	Encryption Method	Key Management
AWS (RDS)	AES-256 via AWS KMS	Customer-managed CMK, auto-rotated annually
Self-hosted (Kubernetes)	LUKS2 full-disk encryption	Key stored in Vault, injected via Kubernetes Secret
Development	PostgreSQL page-level encryption (pg_tde extension)	Local key file

All database connections use TLS (mode `verify-full`), ensuring data is encrypted in transit between the application and the database:

```
dsn := "postgres://helix:password@db.helix-ota.io:5432/helix_ota?
        sslmode=verify-full&sslrootcert=/etc/ssl/certs/ca-
        cert.pem"
```

8.3 Secret Management (Vault Integration)

All secrets are stored in HashiCorp Vault and injected into the application at runtime. No secrets are stored in environment variables, configuration files, or container images.

Secret Type	Vault Path	Rotation
Database credentials	`secret/data/helix-ota/database`	90 days (auto-rotated by Vault)
JWT signing key	`secret/data/helix-ota/jwt`	180 days
TOTP encryption key	`secret/data/helix-ota/totp`	365 days
API key hashing pepper	`secret/data/helix-ota/apikey`	365 days
Download URL HMAC key	`secret/data/helix-ota/download`	90 days
Artifact signing key	`transit/keys/artifact-signing`	Never extracted from HSM

```
package config

import (
    "github.com/hashicorp/vault/api"
)

// VaultSecretProvider loads secrets from HashiCorp Vault at
        runtime.
type VaultSecretProvider struct {
    client *api.Client
}

// LoadDatabaseCredentials fetches DB credentials from Vault.
func (p *VaultSecretProvider) LoadDatabaseCredentials(ctx
        context.Context) (dsn string, err error) {
    secret, err := p.client.Logical().Read("secret/data/helix-ota/
        database")
    if err != nil {
        return "", fmt.Errorf("read database secret from vault:
        %w", err)
    }

    data := secret.Data["data"].(map[string]interface{})
    username := data["username"].(string)
    password := data["password"].(string)
    host := data["host"].(string)
    port := data["port"].(string)
    dbname := data["database"].(string)

    dsn = fmt.Sprintf(
        "postgres://%s:%s@%s:%s/%s?sslmode=verify-full",
```

```
        username, password, host, port, dbname,
    )
    return dsn, nil
}
```

8.4 API Rate Limiting

Rate limiting is enforced at two levels: the API gateway (global) and the application middleware (per-identity).

Endpoint Category	Limit	Key	Scope
Login	10 req/min	IP address	Global
Device check-in	60 req/min	Device ID (mTLS CN)	Per device
Artifact download	100 req/min	Device ID or User ID	Per identity
Artifact upload	10 req/hour	User ID	Per user
Telemetry ingestion	120 req/min	Device ID	Per device
Dashboard API (read)	200 req/min	User ID	Per user
Dashboard API (write)	60 req/min	User ID	Per user

```
package ratelimit

import (
    "context"
    "net/http"
    "time"

    "github.com/go-redis/redis/v8"
)

// RedisRateLimiter implements a distributed token bucket rate
        limiter
// backed by Redis for multi-instance coordination.
type RedisRateLimiter struct {
    client *redis.Client
}

// RateLimitMiddleware enforces per-key rate limiting using a
        sliding window
// algorithm implemented in Redis Lua for atomicity.
func (rl *RedisRateLimiter) RateLimitMiddleware(
    keyFunc func(r *http.Request) string,
    limit int,
    window time.Duration,
) func(http.Handler) http.Handler {
```

```
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
            key := fmt.Sprintf("ratelimit:%s:%s", keyFunc(r),
        window)

            allowed, remaining, resetAt, err := rl.slidingWindow(
                r.Context(), key, limit, window,
            )
            if err != nil {
                // Fail open: if Redis is down, allow the request
                // but log the error for monitoring
                next.ServeHTTP(w, r)
                return
            }

            // Set rate limit headers
            w.Header().Set("X-RateLimit-Limit", fmt.Sprintf("%d",
        limit))
            w.Header().Set("X-RateLimit-Remaining",
        fmt.Sprintf("%d", remaining))
            w.Header().Set("X-RateLimit-Reset", fmt.Sprintf("%d",
        resetAt.Unix()))

            if !allowed {
                w.Header().Set("Retry-After", fmt.Sprintf("%d",
        resetAt.Sub(time.Now()).Seconds()))
                writeAPIError(w, http.StatusTooManyRequests,
        "RATE_LIMIT_EXCEEDED",
                    "Rate limit exceeded. Please retry after the
        Reset time.")
                return
            }

            next.ServeHTTP(w, r)
        })
    }
}
```

8.5 Input Validation and SQL Injection Prevention

SQL injection is prevented through three independent controls:

1. **Parameterized queries exclusively.** All database access uses the `pgx` driver's parameterized query interface. No string concatenation is used to build SQL queries. The repository layer abstracts all query construction.

2. **Input validation with allow-lists.** All API inputs are validated using the `go-playground/validator` package with strict rules. Fields like `device_id` are validated against `^[a-zA-Z0-9_-]+$` (allow-list), not against a list of "dangerous characters" (deny-list).

3. **Type-safe query builders.** For dynamic queries (filtering, sorting), the repository layer uses a type-safe query builder that escapes all user-provided values:

```
package repository

import (
    "fmt"
    "strings"
)

// DeviceQueryBuilder constructs safe SQL queries with
        parameterized values.
// User input is NEVER interpolated directly into the SQL string.
type DeviceQueryBuilder struct {
    conditions []string
    args       []interface{}
    argIndex   int
}

func (qb *DeviceQueryBuilder) FilterByStatus(status string)
        *DeviceQueryBuilder {
    qb.argIndex++
    qb.conditions = append(qb.conditions,
        fmt.Sprintf("status = $%d", qb.argIndex))
    qb.args = append(qb.args, status)
    return qb
}

func (qb *DeviceQueryBuilder) FilterByGroupID(groupID string)
        *DeviceQueryBuilder {
    qb.argIndex++
    qb.conditions = append(qb.conditions,
        fmt.Sprintf("group_id = $%d", qb.argIndex))
    qb.args = append(qb.args, groupID)
    return qb
}

// SortField validates the sort field against an allow-list.
var allowedSortFields = map[string]string{
    "created_at":  "created_at",
    "last_seen":   "last_seen_at",
    "device_id":   "device_id",
    "status":      "status",
}

func (qb *DeviceQueryBuilder) OrderBy(field string, direction
        string) *DeviceQueryBuilder {
    // Allow-list validation: reject unknown sort fields
    dbField, ok := allowedSortFields[field]
    if !ok {
        dbField = "created_at" // Safe default
    }
    // Allow-list validation: only ASC or DESC
```

```
    if strings.ToUpper(direction) != "ASC" {
        direction = "DESC"
    }
    qb.conditions = append(qb.conditions,
        fmt.Sprintf("ORDER BY %s %s", dbField, direction))
    return qb
}
```

8.6 CORS Configuration

Cross-Origin Resource Sharing (CORS) is configured with the principle of least privilege:

```
package middleware

import "net/http"

func CORSMiddleware(allowedOrigins []string) func(http.Handler)
        http.Handler {
    originSet := make(map[string]bool)
    for _, o := range allowedOrigins {
        originSet[o] = true
    }

    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
            origin := r.Header.Get("Origin")

            if originSet[origin] {
                w.Header().Set("Access-Control-Allow-Origin",
            origin)
                w.Header().Set("Vary", "Origin")
            }
            // No wildcard: if origin is not in the allow-list, no
            CORS headers are set

            w.Header().Set("Access-Control-Allow-Methods", "GET,
            POST, PUT, DELETE, OPTIONS")
            w.Header().Set("Access-Control-Allow-Headers",
                "Authorization, Content-Type, X-API-Key, X-
            Request-ID")
            w.Header().Set("Access-Control-Allow-Credentials",
            "true")
            w.Header().Set("Access-Control-Max-Age", "86400") //
            24 hours preflight cache

            if r.Method == http.MethodOptions {
                w.WriteHeader(http.StatusNoContent)
                return
            }

            next.ServeHTTP(w, r)
        })
```

```
    }
}
```

CORS Policy: - **No wildcard origin:** `Access-Control-Allow-Origin` is never set to `*`. Each request's `Origin` header is checked against an explicit allow-list. - **Credentials allowed:** `Access-Control-Allow-Credentials: true` is set only for matched origins, enabling cookie-based authentication for the dashboard. - **Restricted methods:** Only the HTTP methods used by the API are allowed. `PATCH` is not exposed. - **Restricted headers:** Only the headers used by the API are allowed in `Access-Control-Allow-Headers`.

8.7 Security Headers

All HTTP responses include the following security headers:

```
package middleware

import "net/http"

func SecurityHeadersMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
        // Prevent MIME-type sniffing
        w.Header().Set("X-Content-Type-Options", "nosniff")

        // Prevent clickjacking
        w.Header().Set("X-Frame-Options", "DENY")

        // Enable browser XSS filtering (legacy, but defense-in-
        depth)
        w.Header().Set("X-XSS-Protection", "1; mode=block")

        // Force HTTPS for all future requests (1 year, include
        subdomains)
        w.Header().Set("Strict-Transport-Security",
            "max-age=31536000; includeSubDomains; preload")

        // Control referrer information
        w.Header().Set("Referrer-Policy", "strict-origin-when-
        cross-origin")

        // Content Security Policy for dashboard
        w.Header().Set("Content-Security-Policy",
            "default-src 'self'; script-src 'self'; style-src
        'self' 'unsafe-inline'; "+
            "img-src 'self' data:; connect-src 'self' wss:; frame-
        ancestors 'none'")

        // Permissions policy: deny unnecessary browser features
        w.Header().Set("Permissions-Policy",
            "camera=(), microphone=(), geolocation=(),
        payment=()")

        next.ServeHTTP(w, r)
```

```
    })
}
```

---

9. Compliance & Audit

9.1 Complete Audit Trail for All Operations

Every state-changing operation in the system is logged to the `audit_log` table. The audit log captures:

Field	Description	Example
`user_id`	The user who performed the action	`usr_01HXYZABCDEF`
`action`	The action verb	`create`, `update`, `delete`, `login`, `login_failed`, `pause`, `resume`, `halt`, `rollback`
`resource_type`	The type of resource affected	`artifact`, `rollout`, `device`, `user`, `device_group`
`resource_id`	The UUID of the affected resource	`art_01HART003`
`details`	JSONB with before/after values	`{"field": "status", "old_value": "active", "new_value": "paused"}`
`ip_address`	The source IP of the request	`203.0.113.42`

Key audit events:

Event	Trigger	Details Captured
User login	`POST /api/v1/auth/login`	User ID, IP, success/failure, 2FA status
User login failure	Failed authentication	User ID (if identifiable), IP, failure reason
Token reuse detected	Previously-used refresh token	User ID, session ID, token JTI
Artifact upload	`POST /api/v1/artifacts/upload`	User ID, artifact ID, validation results
Artifact deletion	`DELETE /api/v1/artifacts/{id}`	User ID, artifact ID, reason
Rollout creation	`POST /api/v1/rollouts`	User ID, rollout ID, strategy, group
Rollout pause/resume		

Event	Trigger	Details Captured
	`PUT /api/v1/rollouts/{id}/pause`	User ID, rollout ID, old/new status
Device decommission	`DELETE /api/v1/devices/{id}`	User ID, device ID, reason
User role change	`PUT /api/v1/users/{id}`	Admin ID, target user ID, old/new role
Signing key operation	HSM sign API call	Operation ID, key version, artifact hash

9.2 Immutable Log Storage

The `audit_log` table is protected against tampering:

1. **Database-level protection:** The application database role has `INSERT` and `SELECT` permissions on `audit_log` but not `UPDATE` or `DELETE`. This is enforced by PostgreSQL GRANT/REVOKE:

```
-- Application role can only INSERT and SELECT
GRANT INSERT, SELECT ON audit_log TO helix_app;
REVOKE UPDATE, DELETE, TRUNCATE ON audit_log FROM helix_app;

-- Prevent superuser from accidentally truncating (requires
        explicit DROP)
ALTER TABLE audit_log SET (autovacuum_enabled = false);
```

1. **Database trigger protection:** A trigger prevents any UPDATE or DELETE on the audit log:

```
CREATE OR REPLACE FUNCTION prevent_audit_modification()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'audit_log is append-only: % operation not
        permitted', TG_OP;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_log_no_update
    BEFORE UPDATE ON audit_log
    FOR EACH ROW EXECUTE FUNCTION prevent_audit_modification();

CREATE TRIGGER trg_audit_log_no_delete
    BEFORE DELETE ON audit_log
    FOR EACH ROW EXECUTE FUNCTION prevent_audit_modification();
```

1. **Off-site replication:** Audit logs are replicated in near-real-time to an S3 bucket with Object Lock (WORM — Write Once Read Many) compliance mode. This ensures that even a database administrator with superuser access cannot alter the audit trail without detection.

2. **Hash chaining:** Each audit log entry includes a hash of the previous entry, forming a tamper-evident chain:

```
func (s *AuditService) LogEvent(ctx context.Context, entry
        *AuditEntry) error {
    // Get the hash of the previous entry
    prevHash, err := s.repo.GetLatestAuditHash(ctx)
    if err != nil {
        return fmt.Errorf("get previous audit hash: %w", err)
    }

    // Compute this entry's hash: SHA-256(prev_hash + entry_data)
    entry.PreviousHash = prevHash
    entryData := fmt.Sprintf("%s|%s|%s|%s|%s|%v",
        entry.UserID, entry.Action, entry.ResourceType,
        entry.ResourceID, entry.Details, entry.CreatedAt)
    combined := prevHash + "|" + entryData
    entry.Hash = sha256Hash(combined)

    return s.repo.InsertAuditEntry(ctx, entry)
}
```

9.3 Data Retention Policies

Data Category	Retention Period	Disposal Method
Audit log	7 years (regulatory)	Archived to S3 Glacier after 1 year; deleted after 7 years
Telemetry events	13 months (rolling)	Partition auto-detached and dropped after 13 months
Device records	Active + 90 days after decommission	Hard-deleted after 90-day post-decommission retention
Artifact files	Active + 30 days after deletion from system	S3 lifecycle policy deletes storage objects
User records	Active + 30 days after account deactivation	Hard-deleted after 30-day retention
JWT refresh tokens	7 days (token lifetime)	Auto-cleaned by scheduled job
Session records	Until session expires + 24 hours	Auto-cleaned by scheduled job

9.4 GDPR Considerations for Device Data

Device data in the Helix OTA system may contain personal data under GDPR (e.g., IP addresses, geolocation metadata, device identifiers that can be linked to individuals). The following GDPR compliance measures are implemented:

1. **Lawful basis:** Device data processing is based on **legitimate interest** (providing the OTA service) and **contractual necessity** (delivering updates that the device owner has requested).

2. **Data minimization:** Only data strictly necessary for OTA operations is collected. IP addresses are stored only for security logging and are anonymized after 30 days.

3. **Right to erasure:** Device decommissioning (`DELETE /api/v1/devices/{id}`) triggers a soft delete with 90-day retention for audit purposes, followed by hard deletion. Associated telemetry is anonymized (device_id replaced with a random UUID, IP address set to NULL).

4. **Data portability:** Device history and telemetry data can be exported via `GET /api/v1/devices/{device_id}/history` in JSON format.

5. **Consent management:** Device registration is a conscious act (mTLS enrollment), which constitutes informed consent for data processing.

6. **Data Processing Agreement (DPA):** Organizations deploying Helix OTA must sign a DPA with their cloud provider (AWS/GCP) covering the storage and processing of device data.

9.5 Incident Response Procedure

The incident response procedure follows the NIST SP 800-61 framework:

Severity Levels:

Level	Definition	Example	Response Time
SEV-1 (Critical)	Active exploitation or confirmed data breach	Signing key compromised; malicious artifact deployed to fleet	15 minutes
SEV-2 (High)	Vulnerability with high exploitation potential	Authenticated endpoint found without rate limiting; SQL injection discovered	1 hour
SEV-3 (Medium)	Vulnerability with limited exploitation potential	CORS misconfiguration; expired TLS certificate	4 hours
SEV-4 (Low)			24 hours

Level	Definition	Example	Response Time
	Security hardening opportunity	Missing security header; verbose error message	

SEV-1 Response Playbook (Signing Key Compromise):

1. **Detect:** Anomaly detection triggers alert on unexpected signing key usage pattern or external report of compromised artifact.
2. **Contain (0-15 min):**
   - Revoke the compromised signing key in the HSM
   - Pause all active rollouts
   - Disable artifact download endpoint
   - Notify incident response team via PagerDuty
3. **Assess (15-60 min):**
   - Determine which artifacts were signed with the compromised key
   - Identify which devices received those artifacts
   - Check audit log for unauthorized key access
4. **Remediate (1-4 hours):**
   - Rotate to the backup signing key (pre-provisioned for this scenario)
   - Re-sign all active artifacts with the new key
   - Resume rollouts with the newly signed artifacts
   - Deploy firmware update with new public key to all devices
5. **Recover (4-24 hours):**
   - For devices that received malicious artifacts: trigger forced rollback to previous version
   - Monitor fleet health for anomalies
   - Conduct forensic analysis of compromised HSM access logs
6. **Post-Incident (1-2 weeks):**
   - Root cause analysis (RCA) document
   - Security architecture review to prevent recurrence
   - Update threat model based on lessons learned

---

10. Security Testing

10.1 Penetration Testing Plan

Penetration testing is conducted annually by an independent third party and quarterly by the internal security team.

Scope:

Target	Test Type	Frequency
OTA API (all endpoints)	OWASP API Security Top 10	Quarterly
Dashboard (React SPA)	OWASP Web Top 10	Quarterly
mTLS handshake	Certificate validation bypass, protocol downgrade	Annually
Artifact signing pipeline	Key extraction, signature forgery	Annually
Device client SDK	Certificate pinning bypass, binary patching	Annually
Infrastructure (K8s, PostgreSQL, Redis, S3)	Network penetration, privilege escalation	Annually

Testing Methodology:

1. **Reconnaissance:** Enumerate all API endpoints, identify authentication mechanisms
2. **Authentication testing:** Attempt to bypass JWT validation, forge mTLS certificates, brute-force TOTP
3. **Authorization testing:** Attempt privilege escalation from device role to admin, from viewer to operator
4. **Injection testing:** SQL injection on all query parameters, command injection on artifact metadata, SSRF on download URLs
5. **Cryptographic testing:** Attempt signature forgery with wrong keys, test hash collision resistance, test certificate chain validation
6. **Business logic testing:** Attempt to skip rollout phases, apply updates to wrong device model, create artifacts without signing

10.2 Fuzz Testing for Artifact Parser

The artifact parser (ZIP reader, `payload_properties.txt` parser, `care_map.pb` protobuf parser) is a high-risk attack surface because it processes untrusted binary input. Fuzz testing is integrated into the CI pipeline:

```
// Fuzz artifact structure validation
func FuzzStructureCheck(f *testing.F) {
    // Seed corpus: valid OTA ZIP files of varying sizes
    seedCorpus := []string{
        "testdata/valid_ota_small.zip",
        "testdata/valid_ota_large.zip",
        "testdata/valid_ota_minimal.zip",
    }
    for _, seed := range seedCorpus {
        data, err := os.ReadFile(seed)
        if err != nil {
            f.Fatal(err)
        }
        f.Add(data)
    }
```

```
    f.Fuzz(func(t *testing.T, data []byte) {
        // Write fuzz input to a temp file
        tmpFile, err := os.CreateTemp("", "fuzz-*.zip")
        if err != nil {
            return
        }
        defer os.Remove(tmpFile.Name())
        tmpFile.Write(data)
        tmpFile.Close()

        // Run structure check — must not panic or hang
        checker := &StructureChecker{}
        artifact := &Artifact{
            TargetModel:   "rk3588_opi5max",
            TargetVersion: "15.0.1",
        }
        ctx, cancel := context.WithTimeout(context.Background(),
        5*time.Second)
        defer cancel()

        result := checker.Check(ctx, tmpFile.Name(), artifact)
        // We don't assert on the result — the goal is to find
        panics
        // and resource exhaustion bugs, not logic errors
        _ = result
    })
}
```

Fuzzing targets:

Component	Fuzzer	Time Budget	Coverage Target
ZIP reader	Go native fuzzer (`go test -fuzz`)	4 hours per CI run	90% line coverage
`payload_properties.txt` parser	Go native fuzzer	2 hours per CI run	95% line coverage
`care_map.pb` protobuf parser	Go native fuzzer	2 hours per CI run	95% line coverage
REST API input validation	Go native fuzzer + custom corpus	1 hour per CI run	80% line coverage

10.3 Static Analysis (SAST)

Static Application Security Testing is integrated into the CI pipeline and runs on every pull request:

Tool	Purpose	Configuration
gosec	Go-specific security anti-patterns	

Tool	Purpose	Configuration
		`gosec -severity medium -confidence medium ./...`
golangci-lint	General linting with security-focused rules	Enabled: `gosec`, `govet`, `staticcheck`, `unused`, `ineffassign`
Semgrep	Pattern-based security analysis	Custom rules for SQL injection, crypto misuse, auth bypass
Trivy	Container image vulnerability scanning	Fails CI on HIGH or CRITICAL CVEs in base image
npm audit	Dashboard dependency vulnerabilities	Fails CI on HIGH or CRITICAL advisories

Custom Semgrep Rules (example):

```yaml
rules:
  - id: go-sql-string-concat
    patterns:
      - pattern: |
          fmt.Sprintf("... $X ...", ...)
      - metavariable-pattern:
          metavariable: $X
          pattern-regex: "(SELECT|INSERT|UPDATE|DELETE|WHERE)"
    message: "Potential SQL injection via string concatenation.
        Use parameterized queries."
    severity: ERROR
    languages: [go]

  - id: go-crypto-weak-hash
    patterns:
      - pattern: sha1.New()
      - pattern: md5.New()
    message: "Weak hash algorithm detected. Use SHA-256 or
        stronger."
    severity: WARNING
    languages: [go]

  - id: go-tls-insecure
    patterns:
      - pattern: InsecureSkipVerify: true
    message: "TLS verification disabled. Never use
        InsecureSkipVerify in production."
    severity: ERROR
    languages: [go]
```

10.4 Dependency Vulnerability Scanning

Dependency scanning runs on every CI build and nightly:

Tool	Frequency	Action on Finding
Go Vulnerability Database (`govulncheck`)	Every PR + nightly	Block merge on HIGH/CRITICAL; auto-create issue on MEDIUM
Snyk	Nightly	Auto-create Jira ticket; block deploy if fix available and not applied within 7 days
Trivy (container)	Every build	Block deployment on CRITICAL; warn on HIGH
Dependabot	Continuous	Auto-create PR for dependency updates with CVE fixes

```
# CI step: Go vulnerability check
govulncheck ./...
if [ $? -ne 0 ]; then
    echo "Vulnerabilities found. Blocking merge."
    exit 1
fi
```

10.5 Certificate Validation Testing

Certificate validation is tested exhaustively because a failure in this code path can completely undermine mTLS authentication:

```
package auth_test

import (
    "crypto/rand"
    "crypto/rsa"
    "crypto/tls"
    "crypto/x509"
    "crypto/x509/pkix"
    "math/big"
    "testing"
    "time"
)

func TestMTLSRejectsExpiredCertificate(t *testing.T) {
    cert, key := generateTestCertificate(t, func(template
        *x509.Certificate) {
        template.NotBefore = time.Now().Add(-2 * time.Hour)
        template.NotAfter = time.Now().Add(-1 * time.Hour) //
        Already expired
        template.Subject = pkix.Name{
            CommonName:         "dev_test001.helix-ota.io",
            OrganizationalUnit: []string{"device"},
        }
    })
```

```
    tlsCert := tls.Certificate{Certificate: [][]byte{cert},
        PrivateKey: key}
    err := validateClientCertificate(tlsCert)
    if err == nil {
        t.Error("expected expired certificate to be rejected")
    }
}

func TestMTLSRejectsCertificateWithoutDeviceOU(t *testing.T) {
    cert, key := generateTestCertificate(t, func(template
        *x509.Certificate) {
        template.Subject = pkix.Name{
            CommonName:         "dev_test001.helix-ota.io",
            OrganizationalUnit: []string{"admin"}, // Wrong OU
        }
    })

    tlsCert := tls.Certificate{Certificate: [][]byte{cert},
        PrivateKey: key}
    err := validateClientCertificate(tlsCert)
    if err == nil {
        t.Error("expected non-device OU certificate to be
        rejected")
    }
}

func TestMTLSRejectsRevokedCertificate(t *testing.T) {
    // Add certificate to CRL before testing
    cert, key := generateTestCertificate(t, func(template
        *x509.Certificate) {
        template.Subject = pkix.Name{
            CommonName:         "dev_revoked001.helix-ota.io",
            OrganizationalUnit: []string{"device"},
        }
        template.SerialNumber = big.NewInt(9999) // Marked as
        revoked in test CRL
    })

    tlsCert := tls.Certificate{Certificate: [][]byte{cert},
        PrivateKey: key}
    err := validateClientCertificate(tlsCert)
    if err == nil {
        t.Error("expected revoked certificate to be rejected")
    }
}

func TestMTLSRejectsWrongCN(t *testing.T) {
    cert, key := generateTestCertificate(t, func(template
        *x509.Certificate) {
        template.Subject = pkix.Name{
            CommonName:         "attacker.helix-ota.io", // Does
        not match device_id
            OrganizationalUnit: []string{"device"},
```

```
        }
    })

    tlsCert := tls.Certificate{Certificate: [][]byte{cert},
        PrivateKey: key}
    err := validateClientCertificate(tlsCert)
    if err == nil {
        t.Error("expected certificate with wrong CN to be
        rejected")
    }
}

func TestMTLSAcceptsValidDeviceCertificate(t *testing.T) {
    cert, key := generateTestCertificate(t, func(template
        *x509.Certificate) {
        template.Subject = pkix.Name{
            CommonName:         "dev_valid001.helix-ota.io",
            OrganizationalUnit: []string{"device"},
        }
    })

    tlsCert := tls.Certificate{Certificate: [][]byte{cert},
        PrivateKey: key}
    err := validateClientCertificate(tlsCert)
    if err != nil {
        t.Errorf("expected valid device certificate to be
        accepted, got: %v", err)
    }
}
```

Certificate test matrix:

Test Case	Expected Result
Valid device certificate (correct CN, OU, not expired, not revoked)	ACCEPT
Expired certificate	REJECT
Certificate with OU != “device”	REJECT
Certificate with CN that does not match device_id	REJECT
Certificate signed by untrusted CA	REJECT
Certificate in CRL (revoked)	REJECT
Certificate with future NotBefore (clock skew)	REJECT
Certificate with missing Key Usage	REJECT
Self-signed certificate	REJECT

Appendix A — Security Configuration Reference

A.1 Environment Variables (Injected via Vault, NOT set in shell)

Variable	Purpose	Vault Path
`HELIX_DB_DSN`	PostgreSQL connection string	`secret/data/helix-ota/database`
`HELIX_JWT_SIGNING_KEY`	RSA private key for JWT signing	`secret/data/helix-ota/jwt`
`HELIX_ARTIFACT_SIGNING_KEY_ID`	HSM key ID for artifact signing	`transit/keys/artifact-signing`
`HELIX_VAULT_ADDR`	Vault server address	Kubernetes ConfigMap
`HELIX_VAULT_ROLE`	Vault AppRole for authentication	Kubernetes ConfigMap
`HELIX_TLS_CERT_PATH`	Server TLS certificate	Kubernetes Secret
`HELIX_TLS_KEY_PATH`	Server TLS private key	Kubernetes Secret
`HELIX_CA_CERT_PATH`	Device CA certificate for mTLS validation	Kubernetes ConfigMap
`HELIX_CRL_PATH`	Certificate Revocation List	S3 (refreshed every 5 minutes)

A.2 Security Checklist for Production Deployment

- ☐ TLS 1.3 only — no TLS 1.2 or below
- ☐ mTLS enabled for all device endpoints
- ☐ Certificate pinning compiled into device client
- ☐ RSA-4096 artifact signing configured with HSM backend
- ☐ HashiCorp Vault integrated for all secret management
- ☐ Database encryption at rest enabled (LUKS or RDS encryption)
- ☐ Database TLS (sslmode=verify-full) configured
- ☐ All database queries use parameterized statements
- ☐

- [ ] Rate limiting enabled on all endpoints
- [ ] Security headers middleware applied globally
- [ ] CORS configured with explicit origin allow-list (no wildcard)
- [ ] Container runs as non-root with read-only filesystem
- [ ] Seccomp profile applied to container
- [ ] Kubernetes network policies restrict pod communication
- [ ] Audit log triggers prevent UPDATE and DELETE
- [ ] Audit log replication to S3 Object Lock configured
- [ ] TOTP 2FA enforced for all admin and operator accounts
- [ ] Refresh token rotation with reuse detection enabled
- [ ] CRL auto-refresh configured (5-minute interval)
- [ ] Fuzz testing passing in CI
- [ ] SAST tools (gosec, Semgrep) passing in CI
- [ ] Dependency vulnerability scan passing in CI
- [ ] Penetration test report reviewed and findings remediated

---

End of Document — HELOTA-SEC-001 v1.0.0